

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

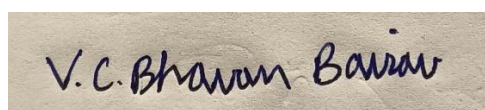
Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I V.C.BHAVAN BAIRAV – 192511369 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison on Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Compare Sequential File Organization and Heap File Organization in Terms of Data Insertion, Deletion, and Retrieval Efficiency

Feature	Sequential File Organization	Heap File Organization
Storage Method	Records stored in sorted order of key	Records stored in no particular order
Insertion	Slow, may require shifting records	Fast, inserted at end of file
Deletion	Costly, may require reorganization	Easy, record can be marked deleted
Retrieval	Very efficient for sequential/range search	Requires linear search
Maintenance	High maintenance	Low maintenance
Example	Banking records sorted by account number	Log files and transaction records

Conclusion: Sequential files are suitable for frequent searching and reporting, whereas heap files are preferred when insertions occur frequently.

Q2. Differentiate Between Clustered Indexing and Non-Clustered Indexing with Suitable Database

Feature	Clustered Index	Non-Clustered Index
Data Storage	Data physically stored in index order	Index stored separately from data
Number per Table	Only one	Multiple allowed
Search Speed	Faster for range queries	Faster for specific lookups
Storage Overhead	Less	More
Example	Student table sorted by Roll No	Index on Student Name

Example:

```
MySQL 8.0 Command Line Client
mysql> CREATE TABLE Student (
  -> RollNo INT NOT NULL,
  -> Name VARCHAR(50),
  -> Department VARCHAR(50),
  -> Marks INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO Student (RollNo, Name, Department, Marks)
  -> VALUES
  -> (101, 'Arun', 'CSE', 85),
  -> (102, 'Meena', 'ECE', 90),
  -> (103, 'Kiran', 'CSE', 78),
  -> (104, 'Ravi', 'IT', 88),
  -> (105, 'Priya', 'CSE', 92);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> SELECT * FROM Student;
+----+-----+-----+-----+
| RollNo | Name | Department | Marks |
+----+-----+-----+-----+
| 101 | Arun | CSE | 85 |
| 102 | Meena | ECE | 90 |
| 103 | Kiran | CSE | 78 |
| 104 | Ravi | IT | 88 |
| 105 | Priya | CSE | 92 |
+----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> CREATE INDEX idx_roll
  -> ON Student(RollNo);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> CREATE INDEX idx_name
  -> ON Student(Name);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
```

```
MySQL 8.0 Command Line Client
mysql> SHOW INDEX FROM Student;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| student | 1 | idx_roll | 1 | RollNo | A | 5 | NULL | NULL | NULL | BTREE | | | YES | NULL |
| student | 1 | idx_name | 1 | Name | A | 5 | NULL | NULL | YES | BTREE | | | YES | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.03 sec)

mysql>
mysql> SELECT *
  -> FROM Student
  -> WHERE RollNo BETWEEN 101 AND 104;
+----+-----+-----+-----+
| RollNo | Name | Department | Marks |
+----+-----+-----+-----+
| 101 | Arun | CSE | 85 |
| 102 | Meena | ECE | 90 |
| 103 | Kiran | CSE | 78 |
| 104 | Ravi | IT | 88 |
+----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT *
  -> FROM Student
  -> WHERE Name = 'Kiran';
+----+-----+-----+-----+
| RollNo | Name | Department | Marks |
+----+-----+-----+-----+
| 103 | Kiran | CSE | 78 |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> EXPLAIN
  -> SELECT *
  -> FROM Student
  -> WHERE RollNo BETWEEN 101 AND 104;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Student | NULL | range | idx_roll | idx_roll | 4 | NULL | 4 | 100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

```
mysql> EXPLAIN
  -> SELECT *
  -> FROM Student
  -> WHERE Name = 'Kiran';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Student | NULL | ref | idx_name | idx_name | 203 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
```

Conclusion: Clustered indexing improves range-based retrieval, while non-clustered indexing supports multiple search attributes.

Q3. Compare Primary Indexing and Secondary Indexing Based on Storage Requirements and Search Performance

Feature	Primary Index	Secondary Index
Based On	Primary key	Non-key attribute
Storage Space	Less	More
Number Allowed	One	Multiple
Search Performance	Very fast	Fast but requires extra access
File Ordering	Data file sorted on primary key	Data file need not be sorted

Example:

- Primary Index on StudentID
- Secondary Index on Department

Conclusion: Primary indexing requires less space and offers better performance, while secondary indexing provides flexible access paths.

Q4. Analyze the Differences Between Static Hashing and Dynamic Hashing in Databases

Feature	Static Hashing	Dynamic Hashing
Bucket Count	Fixed	Variable
Overflow Handling	Overflow chains	Bucket splitting
Performance	Degrades as file grows	Maintains efficiency
Memory Usage	Fixed	Dynamic
Scalability	Poor	Excellent

Example:

Static Hash Function

$h(\text{key}) = \text{key} \bmod 10$

Dynamic Hashing

- Extendible Hashing
- Linear Hashing

Conclusion: Dynamic hashing is better for growing databases due to automatic expansion.

Q5. Compare B-Tree Indexing and B+ Tree Indexing for Large-Scale Database Applications

Feature	B-Tree	B+ Tree
Data Storage	Internal and leaf nodes	Only leaf nodes
Search	Faster for exact search	Better for range queries
Sequential Access	Difficult	Easy through linked leaves
Fan-Out	Lower	Higher
Database Usage	Less common	Widely used

Example:

Databases such as MySQL and Oracle use B+ Trees for indexing.

Conclusion: B+ Trees are preferred because they provide efficient searching and range query performance.

Q6. Differentiate Linear Hashing and Extendible Hashing in Terms of Bucket Management and Scalability

Feature	Linear Hashing	Extendible Hashing
Directory	Not required	Required
Expansion	Gradual bucket split	Directory doubling
Memory Overhead	Low	Higher
Search Speed	Good	Very Good
Scalability	High	Very High

Example:

- Linear Hashing splits buckets sequentially.
- Extendible Hashing doubles directory size when overflow occurs.

Conclusion: Extendible hashing provides better lookup performance, while linear hashing uses less memory.

Q7. Compare Dense Indexing and Sparse Indexing with Respect to Memory Usage and Access Speed.

Feature	Dense Index	Sparse Index
Index Entries	One entry per record	One entry per block
Memory Requirement	High	Low
Search Speed	Faster	Slightly slower
Maintenance Cost	High	Low
Storage Overhead	Large	Small

Example:

Dense Index

- 101 → Block1
- 102 → Block2
- 103 → Block3

Sparse Index

- 101 → Block1
- 201 → Block2
- 301 → Block3

Conclusion: Dense indexing offers faster access but consumes more storage, while sparse indexing saves space.

Q8. Analyze the Advantages and Limitations of Indexed Sequential File Organization Versus Direct File Organization

Feature	Indexed Sequential	Direct File Organization
Access Type	Sequential and random	Direct/random
Search Speed	Fast	Very fast
Range Queries	Supported	Poor support
Storage Overhead	Requires index	No index needed
Maintenance	Moderate	Complex hashing management

Advantages of Indexed Sequential

- Supports range queries.
- Sequential processing possible.

Limitations

- Extra index storage.
- Index maintenance required.

Advantages of Direct File Organization

- Very fast exact record retrieval.
- Suitable for transaction processing.

Limitations

- Poor for range searches.
- Overflow handling needed.

Conclusion: Indexed sequential files are better for reporting, whereas direct files are suitable for OLTP systems.

Q9. Compare Single-Level Indexing and Multi-Level Indexing for Efficient Database Retrieval Operations

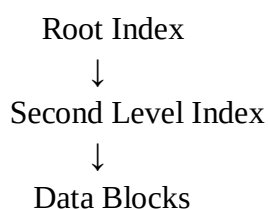
Feature	Single-Level Index	Multi-Level Index
Structure	One index file	Hierarchical indexes
Search Time	More for large files	Less
Storage	Less	More
Scalability	Limited	Excellent
Example	Small database	Large database

Example:

Single Level:

- Index → Records

Multi Level:



Conclusion: Multi-level indexing is preferred for large databases because it significantly reduces search time.

Q10. Evaluate the Performance of Hashing Techniques and Indexing Techniques for Exact-Match and Range Queries in Databases

Criteria	Hashing	Indexing
Exact Match Search	Excellent $O(1)$	Good $O(\log n)$
Range Queries	Poor	Excellent
Ordered Data	Not maintained	Maintained
Insert Performance	Fast	Moderate
Storage Overhead	Low	Higher

Example:

Hash Search

- $h(105)=5$

Index Search

- B+ Tree Search
- $105 \rightarrow \text{Leaf Node}$

Evaluation

- For exact record retrieval, hashing performs best.
- For range queries and sorted access, indexing performs best.

Conclusion: Hashing is ideal for equality searches, while indexing (especially B+ Trees) is preferred for range-based retrieval and analytical workloads.